

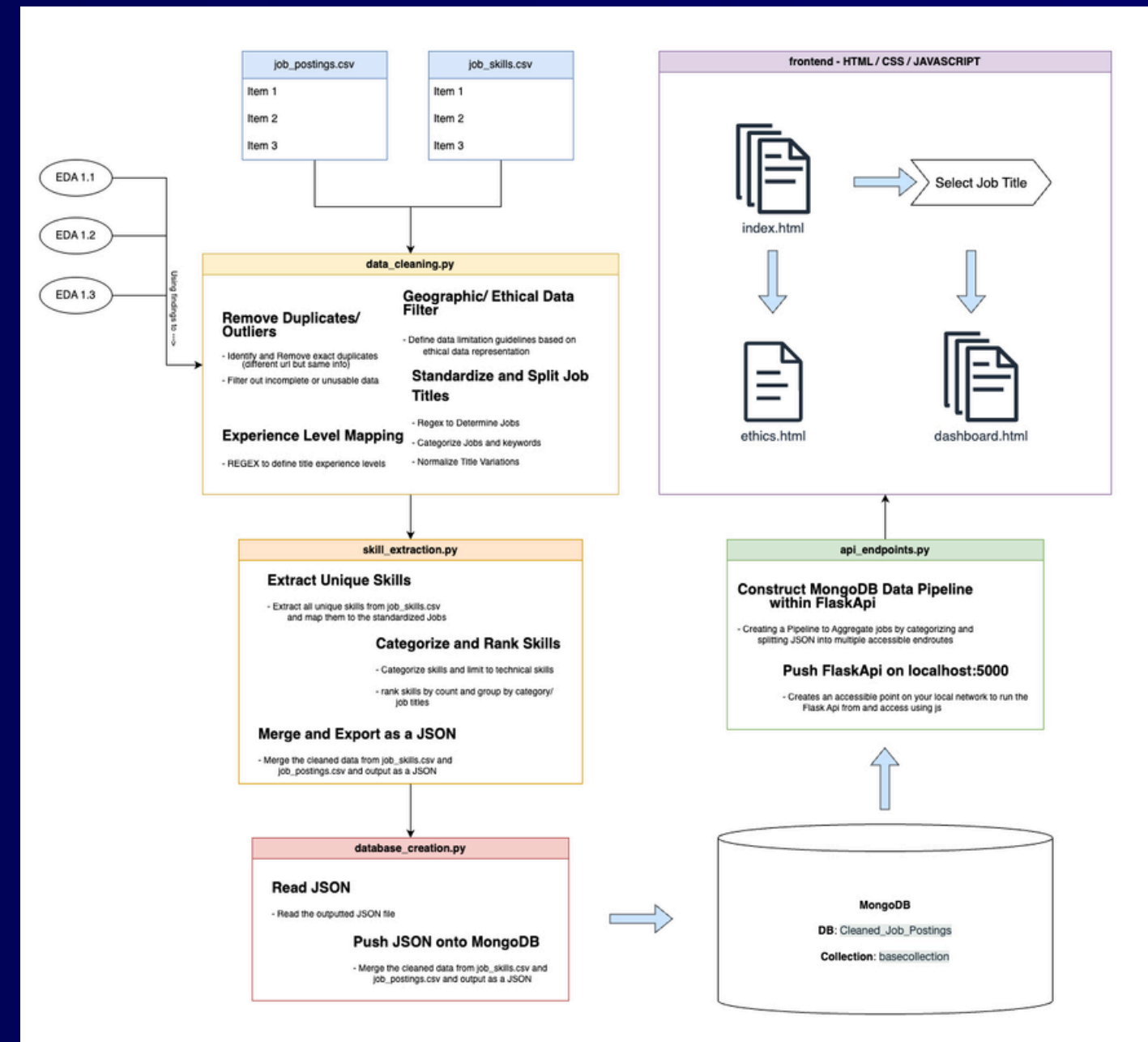
SkillsDashDB

Transforming job postings into career insights

Sean Schallberger
Jose Traboulsi
Karla Lopez
Meghdut Noor

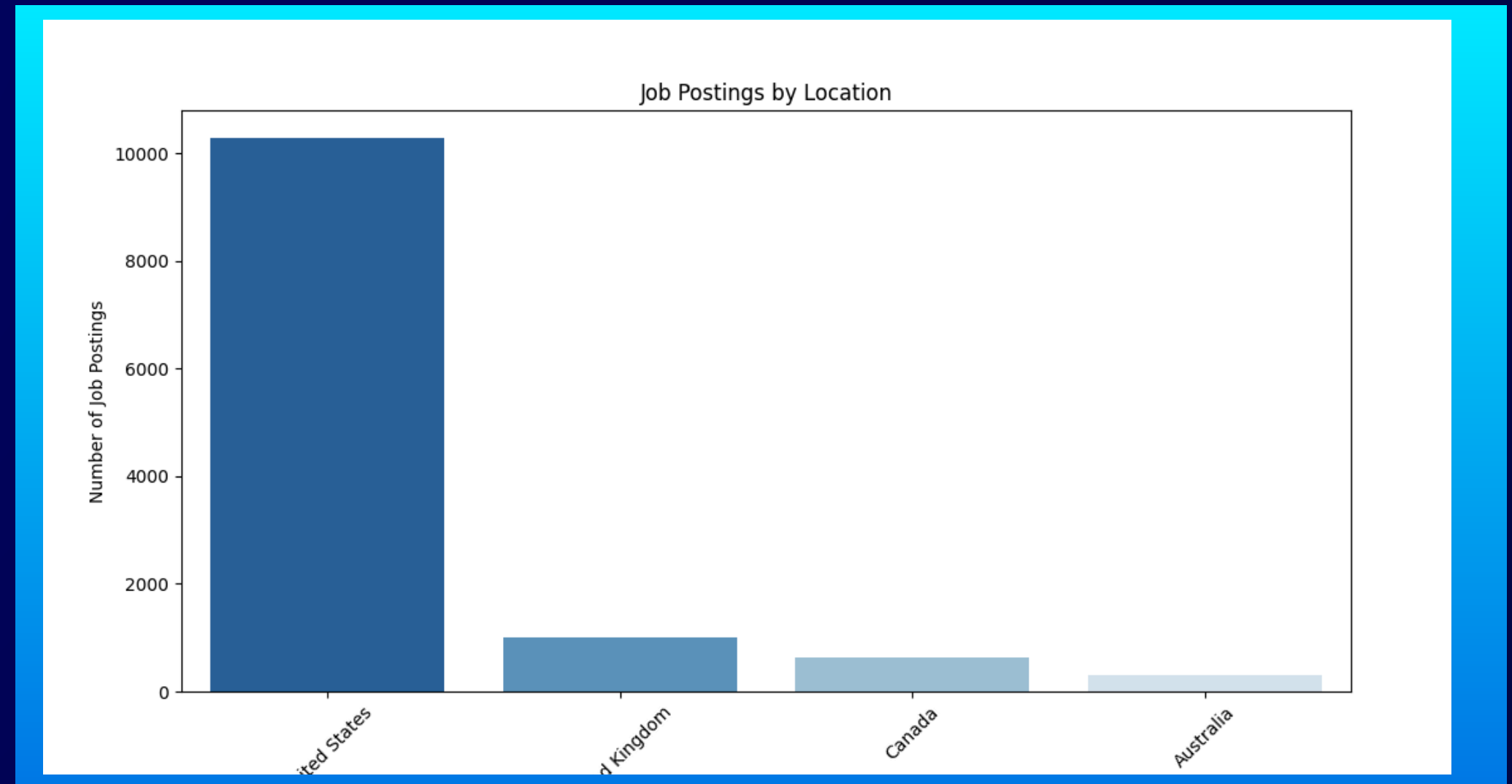
PROJECT INSIGHTS

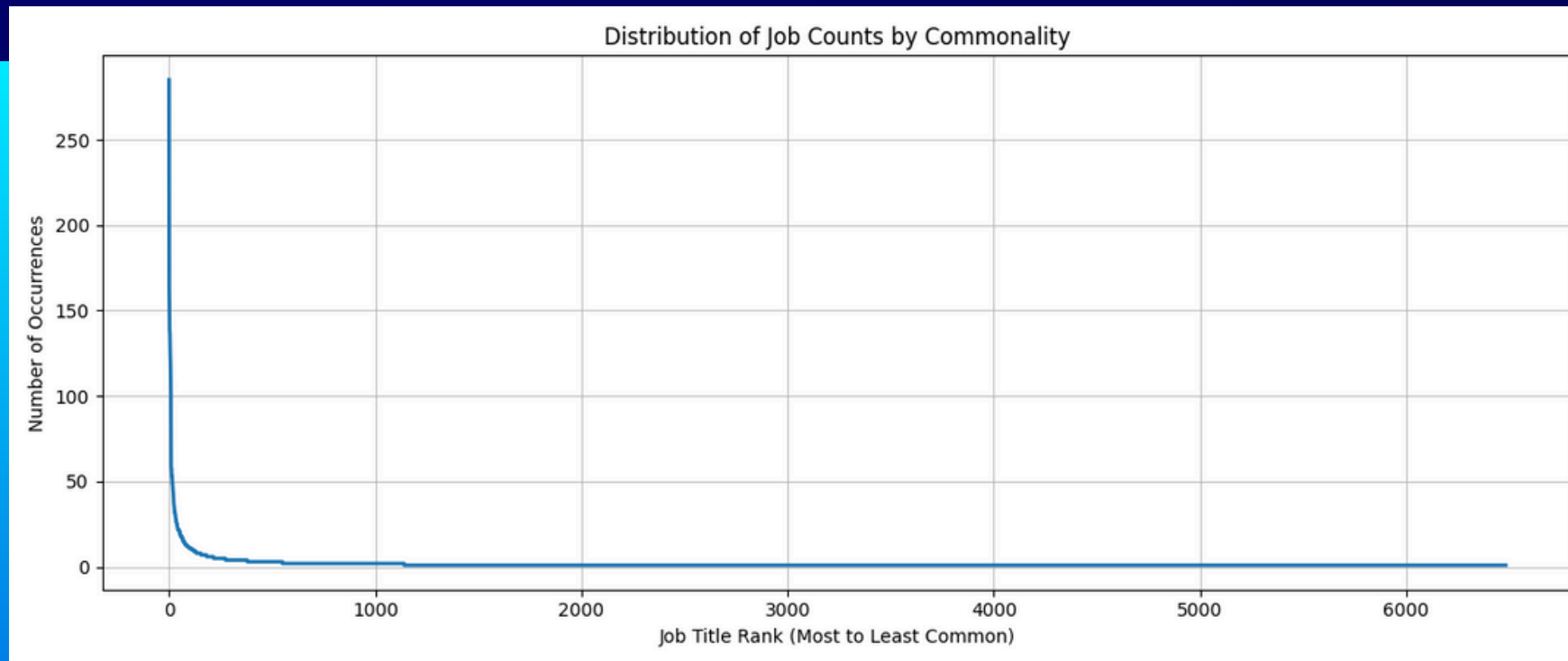
- Problem Statement
- Project Goals
- Target Audience



METHODOLOGY

- **Preliminary Data Cleaning:**
 - Removed Duplicates/ Nulls
 - Limited Scope to the US





'Top 10 Most Common Job Titles:'

	job_title	count
0	Senior Data Engineer	285
1	Senior Data Analyst	163
2	Data Engineer	149
3	Senior MLOps Engineer	138
4	Data Analyst	137
5	Data Scientist	128
6	Lead Data Engineer	123
7	Senior Data Scientist	119
8	Data Architect	111
9	Staff Machine Learning Engineer, Series A	101

'Bottom 10 Least Common Job Titles:'

	job_title	count
6474	Alternative Data Sales	1
6475	Grants and Data Quality Assurance Manager	1
6476	Government Representative 2 Data Coordinator,...	1
6477	Research Data Specialist	1
6478	Medical Laboratory Scientist/MLT	1
6479	Medical Scientist/MLT - Cranberry (FT and PT a...	1
6480	IMLS Graduate Fellow in Objects Conservation	1
6481	SCHOOL DATA MANAGER II 2023-2024 EOY	1
6482	Sr Systems Administrator (Data Management)	1
6483	Medical Technologist, MLS or MLT	1

- **Title Standardization:**

- Raw Job Title Data = 80% Unique
- 14 Structured Representative Job Titles

```
"Data Specialist": r"(?i)(Data Specialist|Data Processing|Data Consultant|D  
"Data Scientist": r"(?i)(Data Scientist|Data Scientists|Data Science Engine  
"Data Analyst": r"(?i)(Data Analyst|Data Analysts|Financial Data Analyst|Bu
```

```
|Data Entry Specialist)\w*[-\s]?",  
Analyst|Data Science Practitioner|Customer Data Scientist)\w*[-\s]?",  
Business Analyst|Data Insights Analyst)\w*[-\s]?",
```

- **Title Standardization:**

- Raw Job Title Data = 80% Unique
- 14 Structured Representative Job Titles

METHODOLOGY

- **Skill Extraction & Ranking:**
 - Used Regex and Conditional techniques.
 - Categorized skills by relevance & frequency.
 - Focus on Top 20 per job category



```
# Standardize job_skills to lowercase and remove extra spaces
merged_df["job_skills"] = merged_df["job_skills"].str.lower().str.strip()

# Get unique skills sorted alphabetically
unique_skills = sorted(merged_df["job_skills"].dropna().unique())

# Save to a file for review (optional)
pd.DataFrame(unique_skills, columns=["Unique Skills"]).to_csv("unique_skills.csv", index=False)

# Display the first few unique skills
unique_skills[:50] # Show first 50 skills for manual review
```

```
# 🤖 Standardizing Data Science & Machine Learning Skills
```

```
if "machine learning" in skill or "ml engineer" in skill:
    return "machine learning"
if "deep learning" in skill:
    return "deep learning"
if "natural language processing" in skill or "nlp" in skill:
    return "natural language processing"
```

```
# 🔥 Cloud & Infrastructure Skills
```

```
if "azure" in skill:
    return "Microsoft Azure"
if "gcp" in skill or "google cloud" in skill:
    return "Google Cloud Platform"
```

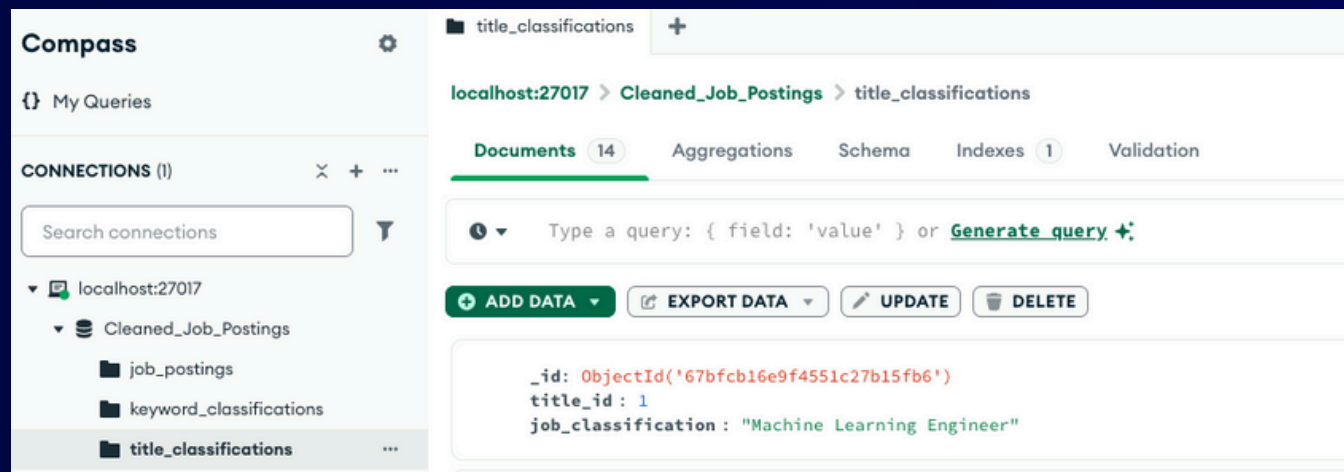
```
# Data Analyst:
```

```
"Data Analysis": r"(?i)data\s*(analysis|analytics)|data\s*analyse",
"Tableau": r"(?i)tableau",
"Data Visualization": r"(?i)data\s+visualization|data\s+visualisation",
"Excel": r"(?i)excel",
"Power BI": r"(?i)power\s*bi",
"Statistics": r"(?i)statistics|statistical",
"Reporting": r"(?i)reporting|reports",
"Teamwork": r"(?i)teamwork|collaboration|team-\s*first\s*mentality",
"Data Mining": r"(?i)data\s+mining|mining",
"Problem Solving": r"(?i)problem\s+solving|troubleshooting",
"Business Intelligence": r"(?i)business\s*intelligence|bi",
"Project Management": r"(?i)project\s*management",
"Data Management": r"(?i)data\s+management",
```


TOOLS AND TECHNIQUES

- **Backend:**

- MongoDB – Stores cleaned job postings & skills.
- Flask/FastAPI – API for skill recommendations.



```
# Create/Connect to the collection
collection = db["mycollection"]
```

```
# Load the JSON file into a Python object
with open("job_postings.json") as f:
    file_data = json.load(f)
```

```
# Insert data into MongoDB
if isinstance(file_data, list): # If JSON is an array
    collection.insert_many(file_data)
else: # If JSON is a single document
    collection.insert_one(file_data)

print("Data inserted successfully!")
```

```
# Import the dependencies
import pandas as pd
import json
from pymongo import MongoClient
```

```
# Read the csv file.
merged_df = pd.read_csv('../data/cleaned_job_postings.csv')
```

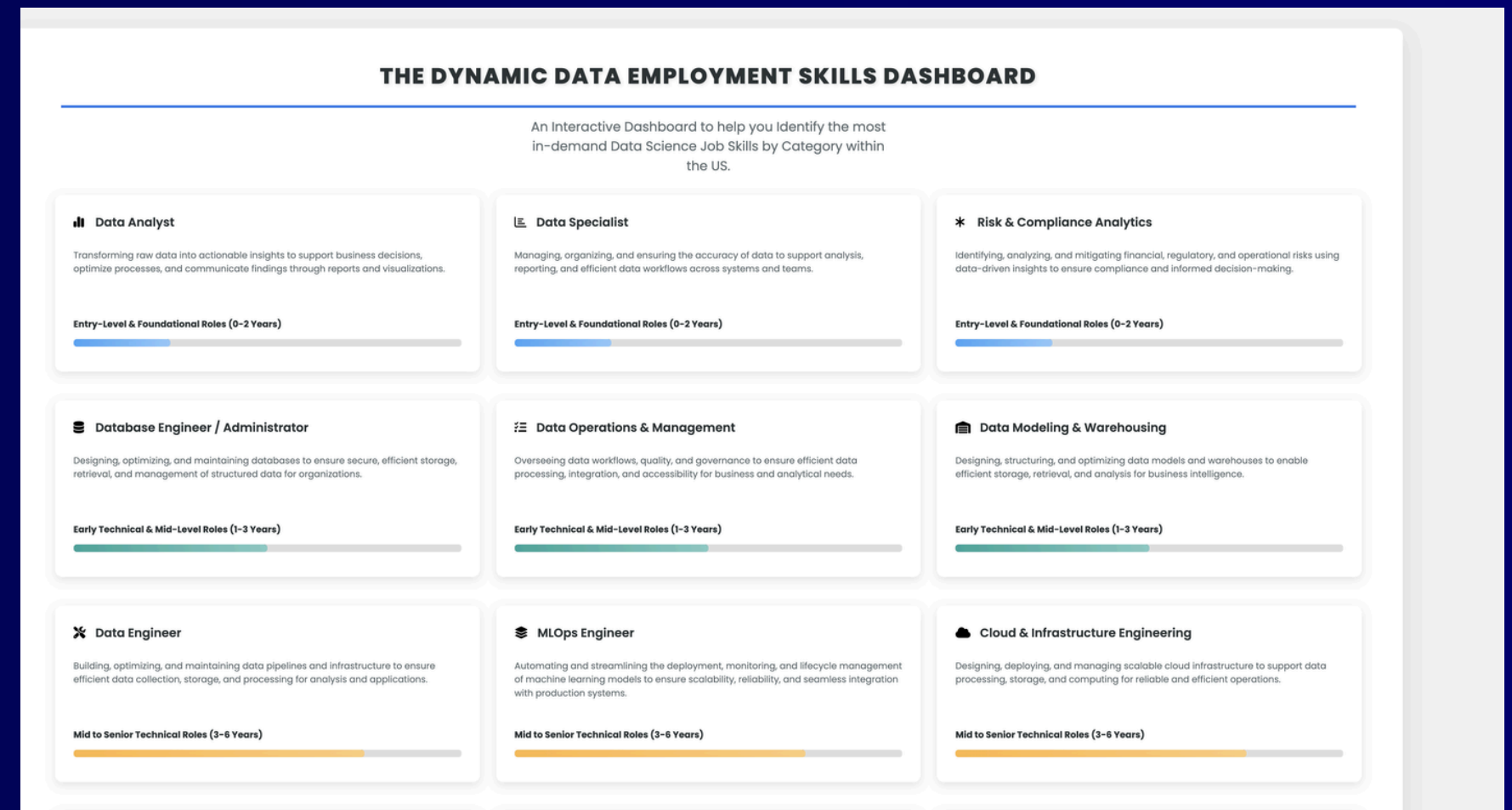
```
# Convert to JSON
job_postings = json.loads(merged_df.to_json(orient='records'))
```

```
# Save JSON to a file
with open("job_postings.json", "w") as json_file:
    json_file.write(json.dumps(job_postings, indent=4))
print("CSV has been converted to JSON successfully!")
```

CSV has been converted to JSON successfully!

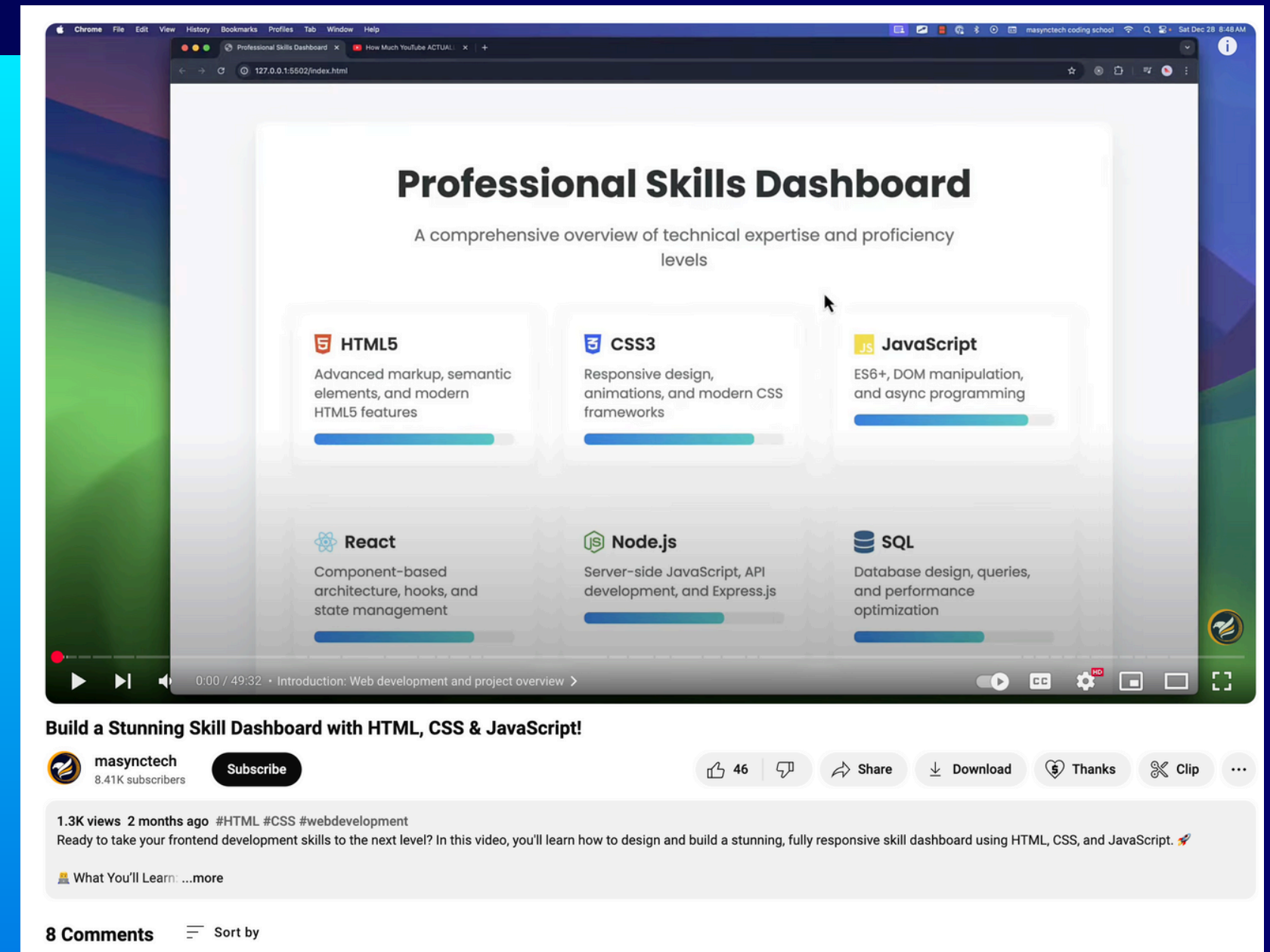
TOOLS AND TECHNIQUES

- **Frontend:**
 - JavaScript-based dashboard – Interactive skill ranking & filtering.



FRONT-END STRUCTURE & DESIGN APPROACH

- **Design:**
 - Simple & Functional
 - Skill Tracker Mini Portfolio Project
 - Category Specific D3.json



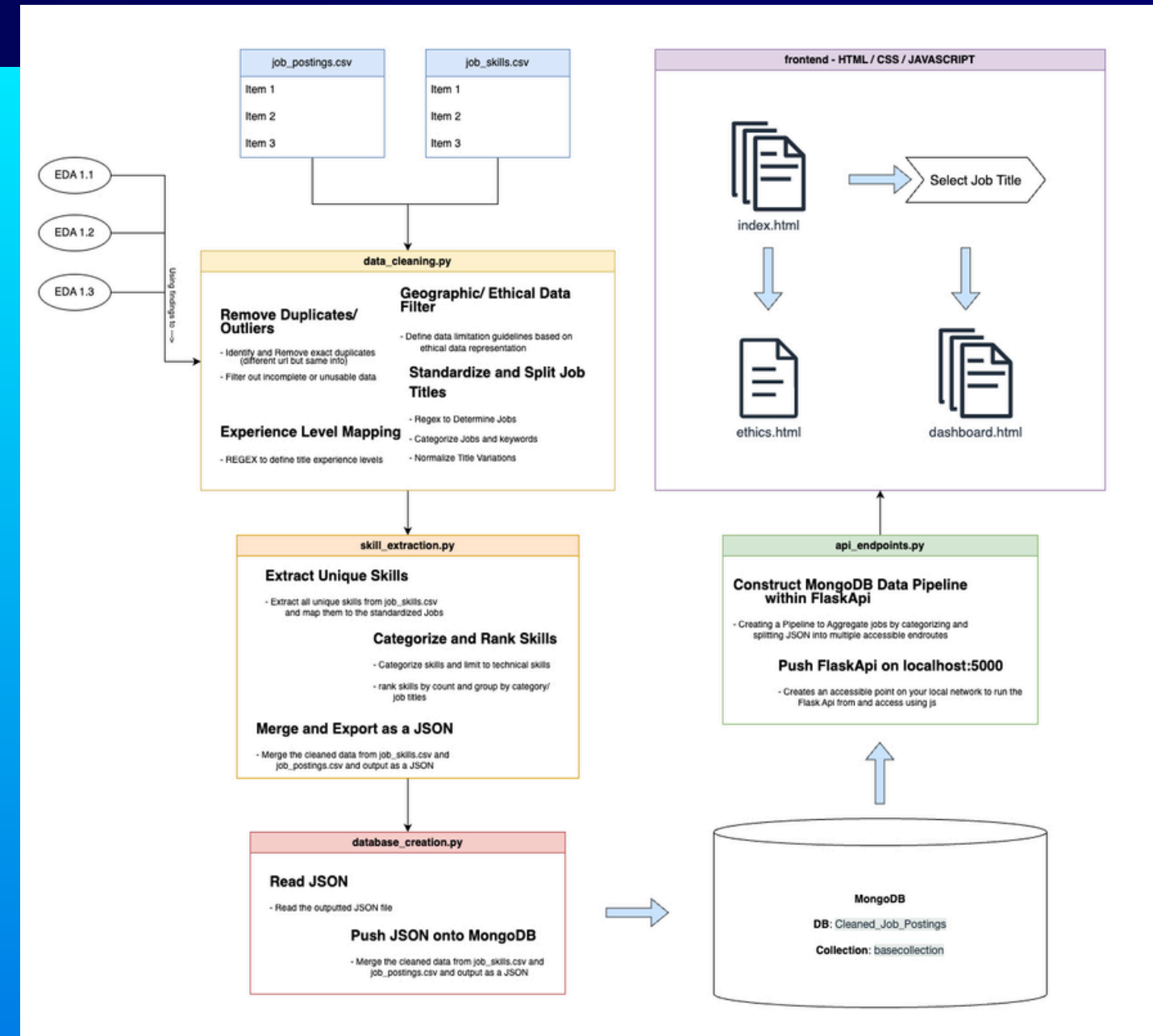
DASHBOARD DEMO

FRONT-END STRUCTURE & DESIGN APPROACH

- **Structure:**
 - Dynamic Linking
- **Executables:**

conda activate dev

python run_scripts.py





CONCLUSION & NEXT STEPS

- Summary
 - Enhancements we would've like to implement.
 - Final Thoughts
-

THANK YOU!

Questions?